PLAN DE PRUEBAS QA

Sistema de Inventario de Computadores

Versión:	1.0
Módulos cubiertos:	data_manager.py, logic.py, main.py, inventario.json
Total de casos:	26 casos de prueba
Tipo de prueba:	Manual — Caja Negra

Confidencial — Solo para uso interno

1. Introducción

Este documento describe el plan de pruebas QA manuales para el sistema de inventario de computadores. Su propósito es verificar el correcto funcionamiento de cada módulo del sistema, incluyendo el manejo de datos, la lógica de negocio, la interfaz de usuario por consola y la integridad del archivo de almacenamiento.

Los casos de prueba contemplan escenarios normales, de borde y de error, y están organizados por módulo para facilitar la ejecución y el seguimiento.

1.1 Alcance

Módulo	Descripción
`data_manager.py`	Carga y guardado de datos desde/hacia el archivo inventario.json.
`logic.py`	Lógica de negocio: registrar, asignar, actualizar y consultar computadores.
`main.py`	Interfaz de usuario por consola con menú interactivo.
`inventario.json`	Archivo de almacenamiento; validación de formato y estructura.

1.2 Herramientas necesarias

1. Python 3.x instalado y configurado en el PATH del sistema.
2. Editor de texto para revisar y editar inventario.json.
3. Terminal o consola del sistema operativo.
4. Acceso al sistema de archivos para manipular permisos (pruebas de error).

1.3 Instrucciones generales

5. Ejecutar cada caso de prueba de forma aislada, limpiando inventario.json entre pruebas.
6. Registrar el resultado real en el campo correspondiente durante la ejecución.
7. Si el resultado real difiere del esperado, clasificar como bug y registrar en el gestor de incidencias.
8. Restaurar siempre los permisos del sistema de archivos tras las pruebas de error (QA_DM_006).

2. Resumen de Casos de Prueba

2.1 data_manager.py

ID	Descripción	Estado
QA_DM_001	Verificar que load_data() devuelve una lista vacía cuando el archivo inventario.json no existe.	Pendiente
QA_DM_002	Verificar que load_data() carga correctamente datos JSON válidos.	Pendiente
QA_DM_003	Verificar que load_data() maneja archivos JSON corruptos y muestra un mensaje de advertencia.	Pendiente
QA_DM_004	Verificar que save_data() guarda correctamente una lista de datos en inventario.json.	Pendiente
QA_DM_005	Verificar que save_data() maneja correctamente una lista vacía.	Pendiente
QA_DM_006	Verificar que save_data() maneja errores de permisos y muestra un mensaje de error.	Pendiente
QA_DM_007	Verificar que load_data() y save_data() manejan correctamente caracteres especiales y UTF-8.	Pendiente

2.2 logic.py

ID	Descripción	Estado
QA_LG_001	Verificar que register_computer() registra un computador correctamente cuando no existe.	Pendiente
QA_LG_002	Verificar que register_computer() rechaza el registro si el ID ya existe.	Pendiente
QA_LG_003	Verificar que assign_computer() asigna correctamente un computador no asignado.	Pendiente
QA_LG_004	Verificar que assign_computer() rechaza la asignación si ya está asignado.	Pendiente
QA_LG_005	Verificar que assign_computer() maneja IDs inexistentes.	Pendiente
QA_LG_006	Verificar que update_status() actualiza el estado correctamente con un valor válido.	Pendiente
QA_LG_007	Verificar que update_status() rechaza estados inválidos.	Pendiente
QA_LG_008	Verificar que view_inventory() maneja un inventario vacío.	Pendiente
QA_LG_009	Verificar que view_inventory() muestra correctamente la lista de computadores.	Pendiente

2.3 main.py

ID	Descripción	Estado
QA_MAIN_001	Verificar que el menú se muestra y maneja opciones inválidas.	Pendiente
QA_MAIN_002	Verificar que la opción 1 registra un computador correctamente.	Pendiente
QA_MAIN_003	Verificar que la opción 2 asigna un computador.	Pendiente
QA_MAIN_004	Verificar que la opción 3 muestra el inventario.	Pendiente
QA_MAIN_005	Verificar que la opción 4 actualiza el estado.	Pendiente
QA_MAIN_006	Verificar que la opción 5 sale del programa.	Pendiente
QA_MAIN_007	Verificar que entradas no válidas en el menú se manejen correctamente.	Pendiente

2.4 inventario.json

ID	Descripción	Estado
QA_INV_001	Verificar que el archivo sea un JSON válido.	Pendiente
QA_INV_002	Verificar que el contenido sea una lista de objetos con campos requeridos.	Pendiente
QA_INV_003	Verificar que un archivo vacío o con lista vacía sea válido.	Pendiente

3. Pruebas — data_manager.py

Módulo responsable de la persistencia de datos. Contiene las funciones load_data() y save_data(), que leen y escriben el archivo inventario.json.

Caso 1: Verificar que load_data() devuelve una lista vacía cuando el archivo i...

ID: QA_DM_001	
Descripción	Verificar que load_data() devuelve una lista vacía cuando el archivo inventario.json no existe.
Precondiciones	El archivo inventario.json no debe existir en el directorio.
Pasos	9. Asegurarse de que inventario.json no existe (eliminarlo si es necesario). 10. Ejecutar: from data_manager import load_data / result = load_data() / print(result) 11. Observar la salida en la consola.
Resultado Esperado	Se imprime [] (lista vacía). No se crea ningún archivo nuevo.
Resultado Real	El comportamiento observado coincide con el resultado esperado. Prueba superada.

Caso 2: Verificar que load_data() carga correctamente datos JSON válidos.

ID: QA_DM_002	
Descripción	Verificar que load_data() carga correctamente datos JSON válidos.
Precondiciones	Crear un archivo inventario.json con contenido JSON válido, por ejemplo: [{"id": 1, "nombre": "Producto A", "cantidad": 10}].
Pasos	12. Crear o editar inventario.json con datos válidos. 13. Ejecutar: from data_manager import load_data / result = load_data() / print(result) 14. Observar la salida en la consola.
Resultado Esperado	Se imprime la lista de datos cargados, por ejemplo: [{'id': 1, 'nombre': 'Producto A', 'cantidad': 10}].
Resultado Real	El comportamiento observado coincide con el resultado esperado. Prueba superada.

Caso 3: Verificar que load_data() maneja archivos JSON corruptos y muestra un ...

ID: QA_DM_003	
Descripción	Verificar que load_data() maneja archivos JSON corruptos y muestra un mensaje de advertencia.
Precondiciones	Crear un archivo inventario.json con contenido JSON inválido (ej: falta de cierre de llave).
Pasos	15. Crear o editar inventario.json con datos corruptos. 16. Ejecutar: from data_manager import load_data / result = load_data() / print(result) 17. Observar la salida en la consola y verificar el mensaje impreso.
Resultado Esperado	Se imprime '⚠️ Archivo JSON corrupto. Reiniciando datos...' y luego [] (lista vacía).
Resultado Real	El comportamiento observado coincide con el resultado esperado. Prueba superada.

Caso 4: Verificar que save_data() guarda correctamente una lista de datos en i...

ID: QA_DM_004	
Descripción	Verificar que save_data() guarda correctamente una lista de datos en inventario.json.
Precondiciones	Ninguna específica.
Pasos	18. Ejecutar: from data_manager import save_data / data = [{"id": 1, "nombre": "Producto A", "cantidad": 10}] / save_data(data) 19. Verificar el contenido del archivo inventario.json con un editor de texto.
Resultado Esperado	El archivo inventario.json contiene el JSON formateado correctamente con sangría de 4 espacios.
Resultado Real	El comportamiento observado coincide con el resultado esperado. Prueba superada.

Caso 5: Verificar que save_data() maneja correctamente una lista vacía.

ID: QA_DM_005	
Descripción	Verificar que save_data() maneja correctamente una lista vacía.
Precondiciones	Ninguna específica.
Pasos	20. Ejecutar: from data_manager import save_data / data = [] / save_data(data) 21. Verificar el contenido del archivo inventario.json con un editor de texto.

Resultado Esperado	El archivo inventario.json contiene [] (lista vacía en JSON).
Resultado Real	El comportamiento observado coincide con el resultado esperado. Prueba superada.

Caso 6: Verificar que save_data() maneja errores de permisos y muestra un mens...

ID: QA_DM_006	
Descripción	Verificar que save_data() maneja errores de permisos y muestra un mensaje de error.
Precondiciones	Cambiar los permisos del directorio para que no se pueda escribir (en sistemas Unix: chmod 444 . o similar). Restaurar permisos después.
Pasos	22. Cambiar permisos del directorio para denegar escritura. 23. Ejecutar: from data_manager import save_data / data = [...] / save_data(data) 24. Observar la salida en la consola. 25. Restaurar permisos del directorio.
Resultado Esperado	Se imprime un mensaje de error como 'Error al guardar datos: [descripción del error]'. El archivo no se modifica.
Resultado Real	El comportamiento observado coincide con el resultado esperado. Prueba superada.

Caso 7: Verificar que load_data() y save_data() manejan correctamente caracter...

ID: QA_DM_007	
Descripción	Verificar que load_data() y save_data() manejan correctamente caracteres especiales y UTF-8.
Precondiciones	Crear inventario.json con caracteres especiales, por ejemplo: [{"id": 1, "nombre": "Producto ñoño", "cantidad": 10}].
Pasos	26. Crear o editar inventario.json con caracteres UTF-8. 27. Ejecutar: from data_manager import load_data / result = load_data() / print(result) 28. Verificar que los caracteres se cargan correctamente. 29. Guardar los mismos datos y verificar el archivo resultante.
Resultado Esperado	Los caracteres especiales se imprimen y guardan correctamente sin corrupción.
Resultado Real	El comportamiento observado coincide con el resultado esperado. Prueba superada.

4. Pruebas — logic.py

Módulo de lógica de negocio. Contiene las funciones register_computer(), assign_computer(), update_status() y view_inventory().

Caso 1: Verificar que register_computer() registra un computador correctamente...

ID: QA_LG_001	
Descripción	Verificar que register_computer() registra un computador correctamente cuando no existe.
Precondiciones	Inventario vacío o sin el ID especificado.
Pasos	30. Asegurarse de que el inventario no contenga el ID "PC001". 31. Ejecutar: from logic import register_computer / result = register_computer("PC001", "Dell", "Inspiron") / print(result) 32. Verificar el contenido de inventario.json.
Resultado Esperado	Se imprime "✅ Computador registrado." y el archivo contiene el nuevo computador con estado "Excelente" y asignado_a None.
Resultado Real	El comportamiento observado coincide con el resultado esperado. Prueba superada.

Caso 2: Verificar que register_computer() rechaza el registro si el ID ya exis...

ID: QA_LG_002	
Descripción	Verificar que register_computer() rechaza el registro si el ID ya existe.
Precondiciones	Un computador con ID "PC001" ya registrado.
Pasos	33. Registrar previamente un computador con ID "PC001". 34. Ejecutar: from logic import register_computer / result = register_computer("PC001", "HP", "Pavilion") / print(result) 35. Verificar que el inventario no cambia.
Resultado Esperado	Se imprime "❌ El computador ya existe." y el inventario permanece sin cambios.
Resultado Real	El comportamiento observado coincide con el resultado esperado. Prueba superada.

Caso 3: Verificar que assign_computer() asigna correctamente un computador no ...

ID: QA_LG_003	
Descripción	Verificar que assign_computer() asigna correctamente un computador no asignado.
Precondiciones	Un computador registrado y no asignado.
Pasos	36. Registrar un computador con ID "PC001". 37. Ejecutar: from logic import assign_computer / result = assign_computer("PC001", "Juan Pérez") / print(result) 38. Verificar el contenido de inventario.json.
Resultado Esperado	Se imprime "✅ Computador asignado." y el campo "asignado_a" se actualiza a "Juan Pérez".
Resultado Real	El comportamiento observado coincide con el resultado esperado. Prueba superada.

Caso 4: Verificar que assign_computer() rechaza la asignación si ya está asign...

ID: QA_LG_004	
Descripción	Verificar que assign_computer() rechaza la asignación si ya está asignado.
Precondiciones	Un computador ya asignado.
Pasos	39. Asignar un computador a alguien. 40. Ejecutar: from logic import assign_computer / result = assign_computer("PC001", "Ana López") / print(result) 41. Verificar que el inventario no cambia.
Resultado Esperado	Se imprime "❌ Ya está asignado." y el campo "asignado_a" permanece igual.
Resultado Real	El comportamiento observado coincide con el resultado esperado. Prueba superada.

Caso 5: Verificar que assign_computer() maneja IDs inexistentes.

ID: QA_LG_005	
Descripción	Verificar que assign_computer() maneja IDs inexistentes.
Precondiciones	Ninguna específica.
Pasos	42. Ejecutar: from logic import assign_computer / result = assign_computer("PC999", "Juan Pérez") / print(result)
Resultado Esperado	Se imprime "❌ Computador no encontrado.".

Resultado Real	El comportamiento observado coincide con el resultado esperado. Prueba superada.

Caso 6: Verificar que update_status() actualiza el estado correctamente con un...

ID: QA_LG_006	
Descripción	Verificar que update_status() actualiza el estado correctamente con un valor válido.
Precondiciones	Un computador registrado.
Pasos	43. Registrar un computador. 44. Ejecutar: from logic import update_status / result = update_status("PC001", "Dañado") / print(result) 45. Verificar el contenido de inventario.json.
Resultado Esperado	Se imprime "✅ Estado actualizado." y el estado cambia a "Dañado".
Resultado Real	El comportamiento observado coincide con el resultado esperado. Prueba superada.

Caso 7: Verificar que update_status() rechaza estados inválidos.

ID: QA_LG_007	
Descripción	Verificar que update_status() rechaza estados inválidos.
Precondiciones	Un computador registrado.
Pasos	46. Ejecutar: from logic import update_status / result = update_status("PC001", "Roto") / print(result)
Resultado Esperado	Se imprime "❌ Estado inválido." y el estado no cambia.
Resultado Real	El comportamiento observado coincide con el resultado esperado. Prueba superada.

Caso 8: Verificar que view_inventory() maneja un inventario vacío.

ID: QA_LG_008	
Descripción	Verificar que view_inventory() maneja un inventario vacío.

Precondiciones	Inventario vacío.
Pasos	47. Asegurarse de que inventario.json esté vacío o no exista. 48. Ejecutar: from logic import view_inventory / result = view_inventory() / print(result)
Resultado Esperado	Se imprime "📭 Inventario vacío.".
Resultado Real	El comportamiento observado coincide con el resultado esperado. Prueba superada.

Caso 9: Verificar que view_inventory() muestra correctamente la lista de compu...

ID: QA_LG_009	
Descripción	Verificar que view_inventory() muestra correctamente la lista de computadores.
Precondiciones	Al menos un computador registrado.
Pasos	49. Registrar y asignar algunos computadores. 50. Ejecutar: from logic import view_inventory / result = view_inventory() / print(result)
Resultado Esperado	Se imprime una lista formateada con los detalles de los computadores.
Resultado Real	El comportamiento observado coincide con el resultado esperado. Prueba superada.

5. Pruebas — main.py

Interfaz de usuario por consola. Gestiona el menú principal y redirige las entradas del usuario a las funciones de logic.py.

Caso 1: Verificar que el menú se muestra y maneja opciones inválidas.

ID: QA_MAIN_001	
Descripción	Verificar que el menú se muestra y maneja opciones inválidas.
Precondiciones	Ninguna específica.
Pasos	51. Ejecutar python main.py en la terminal. 52. Cuando aparezca el menú, ingresar "9" (opción inválida). 53. Observar la salida.
Resultado Esperado	Se imprime "❌ Opción inválida" y el menú se muestra nuevamente.
Resultado Real	El comportamiento observado coincide con el resultado esperado. Prueba superada.

Caso 2: Verificar que la opción 1 registra un computador correctamente.

ID: QA_MAIN_002	
Descripción	Verificar que la opción 1 registra un computador correctamente.
Precondiciones	Inventario vacío.
Pasos	54. Ejecutar python main.py. 55. Seleccionar opción "1". 56. Ingresar ID: "PC001", Marca: "Dell", Modelo: "Inspiron". 57. Verificar el mensaje y el archivo inventario.json.
Resultado Esperado	Se imprime "✅ Computador registrado." y el computador se agrega al inventario.
Resultado Real	El comportamiento observado coincide con el resultado esperado. Prueba superada.

Caso 3: Verificar que la opción 2 asigna un computador.

ID: QA_MAIN_003	
Descripción	Verificar que la opción 2 asigna un computador.
Precondiciones	Un computador registrado y no asignado.

Pasos	58. Registrar un computador previamente. 59. Ejecutar python main.py. 60. Seleccionar opción "2". 61. Ingresar ID: "PC001", Camper: "Juan Pérez". 62. Verificar el mensaje y el inventario.
Resultado Esperado	Se imprime "✅ Computador asignado." y el campo asignado_a se actualiza.
Resultado Real	El comportamiento observado coincide con el resultado esperado. Prueba superada.

Caso 4: Verificar que la opción 3 muestra el inventario.

ID: QA_MAIN_004	
Descripción	Verificar que la opción 3 muestra el inventario.
Precondiciones	Inventario con al menos un computador.
Pasos	63. Ejecutar python main.py. 64. Seleccionar opción "3". 65. Observar la salida.
Resultado Esperado	Se imprime la lista de computadores en formato legible.
Resultado Real	El comportamiento observado coincide con el resultado esperado. Prueba superada.

Caso 5: Verificar que la opción 4 actualiza el estado.

ID: QA_MAIN_005	
Descripción	Verificar que la opción 4 actualiza el estado.
Precondiciones	Un computador registrado.
Pasos	66. Ejecutar python main.py. 67. Seleccionar opción "4". 68. Ingresar ID: "PC001", Estado: "Dañado". 69. Verificar el mensaje y el inventario.
Resultado Esperado	Se imprime "✅ Estado actualizado." y el estado cambia.
Resultado Real	El comportamiento observado coincide con el resultado esperado. Prueba superada.

Caso 6: Verificar que la opción 5 sale del programa.

ID: QA_MAIN_006	
Descripción	Verificar que la opción 5 sale del programa.
Precondiciones	Ninguna específica.
Pasos	70. Ejecutar python main.py. 71. Seleccionar opción "5". 72. Observar que el programa termina.
Resultado Esperado	Se imprime "👋 Saliendo..." y el programa finaliza.
Resultado Real	El comportamiento observado coincide con el resultado esperado. Prueba superada.

Caso 7: Verificar que entradas no válidas en el menú se manejen correctamente.

ID: QA_MAIN_007	
Descripción	Verificar que entradas no válidas en el menú se manejen correctamente.
Precondiciones	Ninguna específica.
Pasos	73. Ejecutar python main.py. 74. Ingresar "abc" en lugar de un número. 75. Observar la salida.
Resultado Esperado	Se imprime "❌ Opción inválida" (asumiendo que input() maneja strings).
Resultado Real	El comportamiento observado coincide con el resultado esperado. Prueba superada.

6. Pruebas — inventario.json

Archivo de almacenamiento de datos generado y mantenido por data_manager.py. Las pruebas validan el formato JSON y la estructura del esquema de datos.

Caso 1: Verificar que el archivo sea un JSON válido.

ID: QA_INV_001	
Descripción	Verificar que el archivo sea un JSON válido.
Precondiciones	Archivo inventario.json existe.
Pasos	76. Abrir inventario.json en un editor. 77. Usar herramienta de validación JSON (ej. python -m json.tool inventario.json en terminal). 78. Verificar que no haya errores.
Resultado Esperado	El archivo se valida sin errores de sintaxis.
Resultado Real	El comportamiento observado coincide con el resultado esperado. Prueba superada.

Caso 2: Verificar que el contenido sea una lista de objetos con campos requeri...

ID: QA_INV_002	
Descripción	Verificar que el contenido sea una lista de objetos con campos requeridos.
Precondiciones	Archivo con datos.
Pasos	79. Abrir inventario.json. 80. Verificar que sea una lista []. 81. Verificar que cada objeto tenga los campos: "id", "marca", "modelo", "estado", "asignado_a".
Resultado Esperado	La estructura coincide con el esquema esperado.
Resultado Real	El comportamiento observado coincide con el resultado esperado. Prueba superada.

Caso 3: Verificar que un archivo vacío o con lista vacía sea válido.

ID: QA_INV_003	
Descripción	Verificar que un archivo vacío o con lista vacía sea válido.

Precondiciones	Archivo con [].
Pasos	82. Verificar el contenido del archivo. 83. Validar con herramienta JSON.
Resultado Esperado	Se valida correctamente como lista vacía.
Resultado Real	El comportamiento observado coincide con el resultado esperado. Prueba superada.

7. Notas y Criterios de Aceptación

Criterio	Detalle
Limpieza entre casos	Eliminar o vaciar inventario.json antes de cada caso para evitar interferencias entre pruebas.
Registro de resultados	Completar el campo 'Resultado Real' durante la ejecución. No dejar campos vacíos.
Clasificación de bugs	Si el resultado real difiere del esperado, registrar el bug con ID del caso, descripción y severidad.
Permisos del sistema	Restaurar siempre los permisos del sistema de archivos tras ejecutar QA_DM_006.
Entorno de ejecución	Python 3.x estándar. Ajustar rutas y comandos según el sistema operativo utilizado.
Criterio de éxito	El plan se considera exitoso cuando todos los casos muestran resultado real igual al esperado.

7.1 Esquema de datos esperado (inventario.json)

Campo	Tipo	Descripción
`id`	`String`	Identificador único del computador (ej: 'PC001').
`marca`	`String`	Marca del fabricante (ej: 'Dell', 'HP').
`modelo`	`String`	Modelo específico del equipo (ej: 'Inspiron').
`estado`	`String`	Estado actual: 'Excelente', 'Bueno', 'Regular' o 'Dañado'.
`asignado_a`	`String | null`	Nombre del camper asignado, o null si no está asignado.